PRAHARI

The sentinel for privileged access

Demo & Presentation Guide

How to run the live demonstration for the judges

Problem Statement 1 — Privileged Access Misuse & Insider Threat Detection
FinSpark'26 · Bank of Maharashtra
github.com/positromen/FinSpark-26-Trial

This script is verified end-to-end against the running server, the database, and the built UI. Follow the beats in order — the ordering is deliberate.

1. The scorecard — PS1 vs Prahari

Every expected outcome and focus area is live in the product and covered in the demo below.

The problem asks for	Prahari delivers	Shown at
Detect misuse of privileged accounts	Rule engine + session recording + banking fraud gates	Beats 1:15, 3:45
Identify insider threats in real time	Every action scored & enforced before it runs; WebSocket push	Whole demo (2 computers)
AI-driven behavioural analysis	IsolationForest + baselines + *measured* 100% detection, 0 false blocks	Beat 5:30
Risk-based access control	ALLOW / MFA / maker-checker / BLOCK, insider-type aware	Beats 3:45, 5:30
Risk-Based Authentication	Step-up MFA at the login screen itself	Beat 0:30
Protect critical admin systems	Blocks, lockout, JIT access, credential checkout, access review	Beats 2:30, 4:15
Quantum-Proof Cryptography	ML-KEM-768 vault · ML-DSA-65 signed chain · signed reports	Beat 6:45

Verified. 54 automated tests pass; a live walk of this exact script passes every beat against the real API and database; both post-quantum providers (native & pure-Python) verified.

2. Setup

Requirements: Python 3.11+, a browser. **No compiler, no internet at demo time** — the post-quantum layer falls back to pure Python automatically.

Start (always fresh for a demo):

`.\run.ps1 -Reset` — wipes + reseeds the database, trains the baseline, serves on `:8000`

Always `-Reset` before a demo. A previous run may have tampered the audit chain (the Tamper button really edits a record) and locked accounts. `-Reset` restores a clean, green starting state.

`run.ps1` prints two URLs: **This computer** (`http://localhost:8000`) and **Other computers** (`http://<LAN-IP>:8000`).

Two-computer demo (recommended — the "wow")

- **Computer 1 (SOC big screen):** open `localhost:8000`, sign in `soc_admin`, stay on **Overview**. A green **LIVE** dot means the feed is connected.
- **Computer 2 (employee):** same Wi-Fi, open the LAN URL, sign in as the employee accounts.

Everything the employee does appears on the SOC screen instantly — no refresh. If Computer 2 can't connect, allow TCP port 8000 through Windows Firewall on Computer 1.

Accounts (all password `prahari123` · MFA code `246810`)

Login	Who	Use for
`soc_admin`	Meera Nair, SOC analyst	the SOC Console
`rmehta`	Rajesh Mehta, DBA	normal employee / transfer **maker** (banking, JIT, vault)
`dgokhale`	Deepa Gokhale, Officer	the second officer / **checker** who approves or clears transfers
`ext_dsouza`	Kevin D'Souza, dormant vendor	the malicious attacker (challenged at login)
`ext_rao`	Priya Rao, expired vendor	the negligent case

3. The 8-minute judge demo

SOC Console on the big screen (`soc_admin`); the employee machine ready. Read the *Say* lines. **The order of the `rmehta` beats is deliberate** — see the note at 1:15.

0:00 — Open on the SOC

30s · framing

Quiet console, green heatmap, the impact-metrics strip, green LIVE dot.

> ***Say:*** *This is Prahari — the sentinel between a bank's privileged staff and its core systems. Right now it's a normal day: every session watched, the audit chain quantum-sealed.*

0:30 — Risk-based authentication

45s · Risk-Based Authentication

On the employee machine, sign in as `ext_dsouza`. An amber step-up screen appears listing the reasons: *dormant account waking up · access expired 120 days ago · unrecognized network/device.*

> ***Say:*** *A dormant vendor account just woke up — and a correct password alone doesn't get in. Risk-based authentication challenges it at the door.*

Enter `246810`.

> ***Say:*** *Our attacker has phished the one-time code — now watch what happens to him* inside.

(Leave him logged in — he is the attacker at 4:15.)

1:15 — Real banking, a normal employee

75s · detect misuse / protect systems

Second employee window: `rmehta` (straight in — trusted context). On the **banking desk**: transfer **Rs 50,000 → CLEARED**; transfer **Rs 5,00,000 → HELD for maker-checker**; pay "Quick-Cash Holdings" (watch-listed) → red **FRAUD flash**, and point to the SOC: the CRITICAL alert just landed.

> ***Say:*** *This is a real banking floor. Big money needs a second officer; a watch-listed payee stops the money and alerts the SOC in the same second.*

Segregation of duties (optional, 30s). Open **Approvals** as `rmehta`: the held item reads *"you initiated this — another officer must approve"* (no buttons). Sign in as `dgokhale` (the officer), **Approve** it — money moves and the ledger shows a check mark with the checker's name; in **Fraud review**, **Clear** the flagged transfer or **Confirm fraud**.

> ***Say:*** *The maker can never approve their own transfer — the checker must be a different, authorized officer, and every decision is signed into the audit chain naming both people.*

Order matters. Do **JIT + vault next** and the **big export last**. Money transfers add no session risk, so `rmehta` is still "clean" for the access beat. If you run the 1,200-record export *first*, its risk stacks with the escalation and `rmehta` is *blocked* before you can show JIT — correct behaviour, wrong demo order.

2:30 — JIT access + the quantum vault

75s · PAM / protect systems

Still `rmehta`, clean session. On the **Action Console** click **Escalate privilege** — it is **allowed but flagged**; open the why-panel: *"privilege change outside normal grant process."*

> ***Say:*** *On its own that's just noted — but there is a right way to do this.*

Go to **JIT Access**, request a grant ("schema migration, 15 min"); approve it on the SOC under **JIT & Credentials** (the badge is already lit). Back on the console, **Escalate privilege** again — the flag is gone: *"sanctioned by an approved JIT grant."*

> ***Say:*** *No standing privilege: elevation is requested with a reason, approved, time-boxed, and auto-expiring — the engine stands down for exactly that grant, then re-arms when it lapses.*

Open **Credential Vault → Check out** the core-banking root password — the secret appears with a 5-minute countdown.

> ***Say:*** *Secrets live sealed under ML-KEM-768 — quantum-safe. The vault opens only for a trusted session, for five minutes, and leaves a signed receipt.*

3:45 — Proportionate response: the big export

30s · risk-based access control

Last `rmehta` action. On the **Action Console**, keep the target on **core-banking-db** (his home system), export **1,200 records → STEP-UP MFA** pop-up → enter `246810` → allowed. The activity log updates a second later.

> ***Say:*** *Unusual volume for a genuine DBA: the response is proportionate — verify, not block.*

(Keep the target on core-banking-db; the same export on a system outside his role is held for maker-checker instead — also correct, but a different beat.)

4:15 — The attack

75s · real-time detection + response

Back to `ext_dsouza`. **Escalate privilege** → held. Then **bulk export 5,000 records → full-screen red BLOCK**, account locked. The SOC screen lights up on its own: a red session at 100, a flashing CRITICAL alert typed **MALICIOUS**, and the replay showing the export **struck**

through — DENIED. Try the **Credential Vault** as the attacker → **checkout refused**. Log him out and back in → **still locked**.

> ***Say:*** *Caught mid-action, before the data left. The block is server-side, the account stays dead, and even the vault refuses him — and every refusal is signed evidence.*

5:30 — Three insider types + explainable, measured AI

75s · AI behavioural analysis

SOC header buttons: **Compromised** → STEP-UP MFA (new country + device, inhuman pace); **Negligent** → MAKER-CHECKER (*"we never auto-block a careless employee; a human reviews"*). Open **AI Model Insights**: feature bars vs the user's own and peer baselines, the risk trajectory climbing, and **Measured Performance**.

> ***Say:*** *We didn't just build AI — we measured it. On a held-out month of 230 benign sessions: 100% of attacks detected with the correct response and type, zero false blocks, 1.7% false alarms.*

6:45 — Quantum-safe evidence + closing the loop

60s · Quantum-Proof Cryptography

Click **Verify** → green, every signature valid. Click **Tamper** → the chain fails **at the exact entry**, the banner turns red.

> ***Say:*** *An insider who edits the evidence would have to forge a NIST post-quantum signature.*

Click **Evidence pack** on the blocked session.

> ***Say:*** *One click — replay, trajectory, model reasoning, audit extract — sealed with an ML-DSA-65 signature. This is what compliance hands to the RBI.*

Finally **Lock account** on any remaining flagged session; the toast confirms it's sealed to the chain.

7:45 — Close

30s

> ***Say:*** *Six outcomes, all live: detection, real time, measured AI, risk-based control from login to logout, protected admin systems with JIT and a quantum vault, and evidence that can't be silently edited. Prahari runs offline, on any laptop, with 54 passing tests — and everything you saw was two computers talking over this Wi-Fi.*

4. Judge Q&A — quick answers

Real AI or rules?
Both, fused. IsolationForest (scikit-learn) on behavioural history + per-user baselines contributes up to 25 points; the explainable rule engine sets the band. Attribution is on the Model Insights page.

How do you know it works?
Held-out benchmark on an independent seed: 100% detection, correct typing and response on the three attacks; 0 false blocks on 230 benign sessions; 1.7% false-alarm rate. Plus 54 pytest tests.

Why isn't negligence blocked?
Type-aware policy: a careless employee is a control failure to remediate with a human second-check (maker-checker), not an attack — enforced in `decide()` and tested.

What is quantum-proof, exactly?
FIPS 203 ML-KEM-768 seals the vault (the AES key *is* the KEM shared secret — defeats harvest-now-decrypt-later); FIPS 204 ML-DSA-65 signs every audit entry and incident report over a hash chain.

What if liboqs won't install?
A pure-Python provider (kyber-py / dilithium-py) is auto-selected; same algorithms, same tests pass. `GET /pqc/info` shows the active provider.

Can the employee bypass the browser?
No — enforcement is server-side; the API refuses. Blocked stays blocked across logins; challenged actions aren't saved until allowed, so the score can't be gamed.

How does it scale?
Stateless scoring behind a load balancer; SQLite → PostgreSQL is one connection string; WebSocket fans out via Redis/Kafka; the same Event schema ingests real PAM/SIEM feeds.

5. Numbers to quote & troubleshooting

Numbers

54	tests, all passing
11	rules, 3 insider types
<1s	score per action
100 / 0 / 1.7%	detection / false blocks / false alarms
Rs 2L / 10L	maker-checker / auto-fraud
5 / 60 min	vault lease / max JIT grant
KEM-768 / DSA-65	FIPS 203 / 204

If something looks off

Chain shows FAILED at start	you tampered earlier: `-Reset`
Attacker already blocked	previous run: `-Reset`
2nd computer can't connect	allow TCP 8000 in firewall
LIVE dot red	auto-reconnects; polling keeps data fresh
liboqs error elsewhere	none — pure-Python fallback loads

Reset between runs: Ctrl+C the server, then `.\run.ps1 -Reset`.
Fallback: record one clean run the night before and keep it on the laptop *and* a phone.

Prahari — detect with explainable, measured AI plus rules; respond in real time; remove standing privilege; keep the proof quantum-safe.